

# Robotaxi 2026 — Behavior Tree Kapsamlı Proje Rehberi

Bu doküman projemizde ne yaptığımızı, yazdığımız kodları, kullandığımız mimariyi ve her bir parçanın ne işe yaradığını **en temelden** anlatan kapsamlı bir rehberdir.

## 1. BEHAVIOR TREE (BT) NEDİR?

Behavior Tree, robotun **karar mekanizmasıdır**. "Şimdi ne yapmalıyım?" sorusuna cevap verir.

**Gerçek hayat benzetmesi:** Bir şoför düşünün:

- Önce yol açık mı kontrol eder → açıksa sürer
- Kırmızı ışık varsa durur → yeşil olunca devam eder
- Yaya varsa durur → geçince devam eder

BT de aynı mantığı **ağaç yapısında** kodlar. Her karar bir "node" (düğüm) olur.

### Neden BT? (if/else değil?)

```
// if/else ile yazsan:
if (acil_durum) { dur(); }
else if (yaya_var) { dur(); bekle(); }
else if (kirmizi_isik) { dur(); bekle(); }
else if (kavsakta) { yavasla(); don(); }
else { normal_sur(); }
// → 50+ senaryo olunca SPAGETTI KOD olur, bakımı imkansız
```

BT'de her senaryo **bağımsız bir node**. Ekle/çıkar/değiştir — diğerlerini etkilemez.

### BT'nin 3 Temel Dönüş Değeri

Her node çalışınca şu 3 değerden birini döner:

| Değer   | Anlamı                  | Örnek                        |
|---------|-------------------------|------------------------------|
| SUCCESS | "İşim bitti, başarılı"  | Hız ayarlandı ✓              |
| FAILURE | "İşim bitti, başarısız" | Yaya yok (tehlike yok)       |
| RUNNING | "Henüz bitmedi, bekle"  | 15sn bekleme devam ediyor... |

### BT'nin 4 Kontrol Node'u

Bu node'lar çocuklarını (alt node'ları) nasıl çalıştıracağına karar verir:

**Sequence (Sıralı):** Çocukları sırayla çalıştırır. Biri FAILURE dönerse DURUR.

```
<Sequence>          <!-- Hepsi başarılı olmalı -->
  <YavasLa />        <!-- SUCCESS → sonrakine geç -->
  <DurCizgisindeKal /> <!-- SUCCESS → sonrakine geç -->
```

```
<YesilBekle />      <!-- FAILURE → Sequence FAILURE döner, hepsi durur -->
</Sequence>
```

**Fallback (Yedek plan):** Çocukları sırayla dener. Biri SUCCESS dönerse DURUR.

```
<Fallback>          <!-- İlk başarılı olan yeter -->
  <SeritDegistir />  <!-- FAILURE → sonrakini dene -->
  <RotaDegistir />   <!-- SUCCESS → Fallback SUCCESS döner -->
</Fallback>
```

**ReactiveSequence:** Sequence gibi ama her tick'te BAŞTAN kontrol eder (güvenlik için).

**Repeat:** Çocuğunu N kez tekrarlar.

## 2. NODE TÜRLERİ — C++ SINIF HİYERARŞİSİ

BT.CPP v3 kütüphanesinde 3 ana node tipi vardır:

### 2.1 ConditionNode — "Koşul Kontrol"

- **Tek iş:** Bir şeyi kontrol et, SUCCESS veya FAILURE dön
- **RUNNING dönemez!** (anlık kontrol)
- **Yan etkisi yoktur** (hiçbir şeyi değiştirmez)

```
// HPP dosyası (sınıf tanımı):
class HasMoreSegments : public BT::ConditionNode { // ← ConditionNode'dan türetildi
public:
  // Constructor – her node'da AYNI şablon
  HasMoreSegments(const std::string& name, const BT::NodeConfiguration& config)
    : BT::ConditionNode(name, config) {}

  // Port tanımı – bu node hangi verileri alıyor/veriyor
  static BT::PortsList providedPorts() {
    return {
      BT::InputPort<int>("seg_index"), // Blackboard'dan OKU
      BT::InputPort<int>("route_size") // Blackboard'dan OKU
    };
  }

  // tick() – node çalışınca çağrılan FONKSİYON
  BT::NodeStatus tick() override;
};

// CPP dosyası (implementasyon):
BT::NodeStatus HasMoreSegments::tick()
{
  int seg_index = 0, route_size = 0;
  getInput("seg_index", seg_index); // Port'tan oku
  getInput("route_size", route_size); // Port'tan oku

  return (seg_index < route_size)
```

```

    ? BT::NodeStatus::SUCCESS    // Daha segment var
    : BT::NodeStatus::FAILURE;    // Bitti
}

```

## 2.2 SyncActionNode — "Anlık Eylem"

- **Tek iş:** Bir eylem yap ve hemen bitir
- SUCCESS veya FAILURE döner (RUNNING dönemez)
- **Yan etkisi VARDIR** (bir şeyi değiştirir: topic publish, blackboard yaz)

```

// HPP:
class SetMaxSpeed : public BT::SyncActionNode { // ← SyncActionNode'dan türetildi
public:
    SetMaxSpeed(const std::string& name, const BT::NodeConfiguration& config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<double>("speed") };
    }

    BT::NodeStatus tick() override;
};

// CPP:
BT::NodeStatus SetMaxSpeed::tick()
{
    double speed = 0.0;
    if (!getInput("speed", speed)) { // Port'tan oku, başarısızsa FAILURE
        return BT::NodeStatus::FAILURE;
    }

    // Blackboard'a kaydet (BT içi paylaşım)
    config().blackboard->set("current_max_speed", speed);

    // ROS2 topic'e publish et (BT dışı paylaşım)
    auto ros = getRosNode(config()); // Blackboard'dan ROS node al
    if (ros) {
        auto pub = ros->create_publisher<std_msgs::msg::Float64>("/vehicle/max_speed",
10);
        std_msgs::msg::Float64 msg;
        msg.data = speed;
        pub->publish(msg);
    }

    return BT::NodeStatus::SUCCESS; // İş bitti
}

```

## 2.3 StatefulActionNode — "Süreçli Eylem"

- **Birden fazla tick sürer** (RUNNING dönebilir)
- 3 fonksiyon: `onStart()`, `onRunning()`, `onHalted()`

```

// HPP:
class Dwell : public BT::StatefulActionNode { // ← StatefulActionNode
public:
    Dwell(const std::string& name, const BT::NodeConfiguration& config)
        : BT::StatefulActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<double>("min_sec"),
            BT::InputPort<double>("max_sec")
        };
    }

    BT::NodeStatus onStart() override; // İlk tick'te çağrılır
    BT::NodeStatus onRunning() override; // Sonraki tick'lerde çağrılır
    void onHalted() override; // İptal edildiğinde çağrılır

private:
    std::chrono::steady_clock::time_point start_time_; // Başlangıç zamanı
    double wait_duration_sec_ = 15.0;
};

// CPP:
BT::NodeStatus Dwell::onStart() // ← Sadece İLK tick
{
    double min_sec = 15.0, max_sec = 20.0;
    getInput("min_sec", min_sec);
    getInput("max_sec", max_sec);
    wait_duration_sec_ = (min_sec + max_sec) / 2.0;
    start_time_ = std::chrono::steady_clock::now(); // Zamanlayıcı başlat
    return BT::NodeStatus::RUNNING; // "Henüz bitmedi"
}

BT::NodeStatus Dwell::onRunning() // ← Sonraki her tick
{
    auto elapsed = std::chrono::steady_clock::now() - start_time_;
    double sec = std::chrono::duration<double>(elapsed).count();

    if (sec >= wait_duration_sec_) {
        return BT::NodeStatus::SUCCESS; // Süre doldu, bitti
    }
    return BT::NodeStatus::RUNNING; // Henüz dolmadı, devam
}

void Dwell::onHalted() // ← Güvenlik refleksi tetiklenirse
{
    RCLCPP_WARN(btLogger(), "Dwell: bekleme kesildi");
}

```

---

### 3. PORT SİSTEMİ VE BLACKBOARD

#### Blackboard Nedir?

Tüm node'ların **ortak hafızası**. Bir node yazıyor, başka node okuyor.

```
Blackboard (ortak hafıza):
  seg_index = 3
  route_size = 12
  cruise_speed = 1.50
  light_color = "red"
  headlights_on = true
```

#### Port Tipleri

| Port Tipi            | Yön     | Kullanım                         |
|----------------------|---------|----------------------------------|
| InputPort<T>         | Oku     | Blackboard'dan veri AL           |
| OutputPort<T>        | Yaz     | Blackboard'a veri YAZ            |
| BidirectionalPort<T> | Oku+Yaz | Hem oku hem yaz (AdvanceSegment) |

#### XML'de Port Bağlama

```
<!-- {süslü parantez} = Blackboard değişkenine bağla -->
<SetMaxSpeed speed="{cruise_speed}" />
<!-- ↑ cruise_speed Blackboard'dan okunur (1.50) -->

<!-- Sabit değer (süslü parantez yok) -->
<SetMaxSpeed speed="0.30" />
<!-- ↑ Doğrudan 0.30 kullanılır -->
```

### 4. ROS2 ALTYAPISI — bt\_node\_base.hpp

BT node'larımız ROS2 ile nasıl konuşuyor?

```
// bt_node_base.hpp – İki yardımcı fonksiyon

// 1) Logger – tüm node'lar aynı isimle log basar
inline rclcpp::Logger btLogger() {
  return rclcpp::get_logger("segment_bt");
}
// Kullanım: RCLCPP_INFO(btLogger(), "Hız: %.2f", speed);

// 2) ROS Node erişimi – publisher/subscriber oluşturmak için
inline rclcpp::Node::SharedPtr getRosNode(const BT::NodeConfiguration& config) {
  rclcpp::Node::SharedPtr node;
  config.blackboard->get("ros_node", node); // main.cpp'de set edilmişti
  return node;
}
```

```

}
// Kullanım:
// auto ros = getRosNode(config());
// auto pub = ros->create_publisher<std_msgs::msg::Bool>("/topic", 10);

```

## main.cpp — Her şeyi birleştiren dosya

```

int main(int argc, char **argv) {
    rclcpp::init(argc, argv); // ROS2 başlat
    auto ros_node = rclcpp::Node::make_shared("bt_test"); // ROS node oluştur

    BT::BehaviorTreeFactory factory; // BT fabrikası
    robotaxi_bt::registerAllNodes(factory); // 43 node'u kaydet

    auto tree = factory.createTreeFromFile(xml_file); // XML'den ağaç oluştur
    tree.rootBlackboard()->set("ros_node", ros_node); // ROS node'u paylaş

    BT::NodeStatus result = tree.tickRoot(); // Ağacı çalıştır!
}

```

## 5. DOSYA YAPISI VE MODÜLER MİMARİ

### HPP vs CPP Nedir?

- **HPP (Header):** Sınıf tanımı — "Bu node ne yapacak?" (arayüz)
- **CPP (Source):** Gerçek kod — "Nasıl yapacak?" (implementasyon)

```

src/robotaxi_bt/
├── include/robotaxi_bt/          ← HPP dosyaları (tanımlar)
│   ├── bt_node_base.hpp         ← ROS2 erişim altyapısı
│   ├── segment_loop_nodes.hpp  ← ✅ 5 node tanımı
│   ├── vehicle_control_nodes.hpp ← ✅ 4 node tanımı
│   ├── traffic_logic_nodes.hpp  ← ✅ 4 node tanımı
│   ├── mission_utility_nodes.hpp ← ✅ 3 node tanımı
│   ├── stub_nodes.hpp          ← ⌚ 27 stub tanımı
│   └── register_all_nodes.hpp   ← 43 node'u kayıt eden fonksiyon
├── src/                          ← CPP dosyaları (kodlar)
│   ├── main.cpp                ← Programın giriş noktası
│   ├── segment_loop_nodes.cpp  ← ✅ 5 node implementasyonu
│   ├── vehicle_control_nodes.cpp ← ✅ 4 node implementasyonu
│   ├── traffic_logic_nodes.cpp  ← ✅ 4 node implementasyonu
│   ├── mission_utility_nodes.cpp ← ✅ 3 node implementasyonu
│   └── stub_nodes.cpp          ← ⌚ 27 stub implementasyonu
├── behaviour_trees/
│   └── segment_bt.xml           ← Ana BT ağacı (494 satır)
└── CMakeLists.txt              ← Derleme konfigürasyonu

```

## 6. TAMAMLANAN 16 NODE — GRUP GRUP DETAYLI ANALİZ

## GRUP 1: Segment Döngüsü (segment\_loop\_nodes)

Bu grup, rota üzerindeki segmentler arasında döngü yapar.

**HasMoreSegments** — `seg_index < route_size` mi kontrol eder.

- Tip: `ConditionNode`
- Portlar: `seg_index` (oku), `route_size` (oku)
- SUCCESS = daha segment var, FAILURE = bitti

**IsSegmentType** — Aktif segmentin tipini karşılaştırır.

- Tip: `ConditionNode`
- Portlar: `seg_type` (oku), `expected` (oku)
- Mantık: `seg_type == expected` → SUCCESS
- XML'de switch/case gibi kullanılır

**IsTrafficControlActive** — `Tur ≥ 2` mi kontrolü.

- Tip: `ConditionNode`
- Port: `tour` (oku)
- Mantık: Tur 1'de trafik ışıkları sahada yok, bu yüzden algılama kapalı

**AdvanceSegment** — `seg_index++` yapar.

- Tip: `SyncActionNode`
- Port: `seg_index` (`BidirectionalPort` — hem oku hem yaz)

**ClearHandledFlags** — Latch bayraklarını sıfırlar.

- Tip: `SyncActionNode`
- Mantık: `handled_stop_sign`, `handled_traffic_light` vb. false yapar
- Neden: Aynı DUR tabelasında sonsuz döngüyü önler

## GRUP 2: Araç Kontrolü (vehicle\_control\_nodes)

**SetMaxSpeed** — Hız limiti ayarlar.

- Tip: `SyncActionNode`
- Port: `speed` (oku, double)
- Yaptığı: Blackboard'a yazar + `/vehicle/max_speed` topic'ine publish eder

**StopVehicle** — Aracı durdurur.

- Tip: `SyncActionNode`
- Yaptığı: `/cmd_vel` topic'ine sıfır Twist mesajı gönderir

**TurnHeadlights** — Far aç/kapat.

- Tip: `SyncActionNode`
- Port: `state` (oku, "on" veya "off")
- Yaptığı: `/vehicle/headlights` topic'ine Bool publish eder

**Dwell** — Belirli süre bekler (15-20 sn).

- Tip: **StatefulActionNode** (birden fazla tick sürer!)
- Portlar: `min_sec`, `max_sec` (oku)
- Yaşam döngüsü: `onStart`→`RUNNING`→`onRunning`→`RUNNING`→...→`SUCCESS`

## GRUP 3: Trafik Mantığı (traffic\_logic\_nodes)

**IsLightRed** — Işık rengi "red" mi?

- Tip: ConditionNode
- Port: `color` (oku)

**IsRoadSignType** — Levha tipi kontrolü.

- Tip: ConditionNode
- Portlar: `sign_type` , `expected`

**StopAndProceed** — DUR tabelası mantığı.

- Tip: SyncActionNode
- Latch mekanizması: `handled_stop_sign` bayrağı ile aynı segmentte tekrar tetiklenmez

**LogUnknownSign** — Bilinmeyen levhayı loglar.

- Tip: SyncActionNode

## GRUP 4: Görev Yardımcıları (mission\_utility\_nodes)

**RecordMissionPoint** — Görev noktası sayacı artırır (+30 puan).

- Tip: SyncActionNode
- Blackboard'daki `mission_points_reached` sayacını artırır

**RecordParkEntryReached** — Park girişi kaydı (+20 puan).

- Tip: SyncActionNode

**SignalPassengerEvent** — Yolcu olayı bildirir.

- Tip: SyncActionNode
- `/mission/passenger_event` topic'ine "pickup"/"dropoff" publish eder

---

## 7. STUB NODE'LAR — NEDEN STUB?

**Stub = İskelet kod.** Derlenir, çalışır, ama gerçek veri kullanmaz.

Örnek — gerçek vs stub karşılaştırma:

```
// GERÇEK (tamamlanmış) node:
BT::NodeStatus SetMaxSpeed::tick() {
    double speed; getInput("speed", speed);
    auto pub = ros->create_publisher<Float64>("/vehicle/max_speed", 10);
    pub->publish(msg); // ← GERÇEKTEN hız komutu gönderiyor
    return SUCCESS;
}

// STUB node:
BT::NodeStatus PedestrianAhead::tick() {
    // TODO: /perception/pedestrian topic'ini subscribe et
    return FAILURE; // ← Her zaman "yaya yok" diyor (sahte)
}
```



## 27 Stub — 3 Kategori

**Harita Bağımlı (5):** LoadMission, GetCurrentSegment, ReplanRoute, CalculateLaneChange, FindParkingSlot → Beklenen: segment\_map.yaml + GeoJSON dosyası

**Sensör Bağımlı (12):** EmergencyStopRequested, PedestrianAhead, DynamicObstacleAhead, StaticObstacleInLane, TrafficLightAhead, StopSignAhead, vb. → Beklenen: Algılama ekibinin YOLO/Lidar ROS topic'leri

**Nav2/Hareket Bağımlı (10):** FollowLaneSegment, WaitForClear, ExecuteParking, BackUp, Spin, vb. → Beklenen: Nav2 NavigateToPose action server

## 8. XML AĞACI — BÜYÜK RESİM

```
MainTree
├─ SetBlackboard (hız parametreleri)
├─ LoadMission (GeoJSON → rota)
├─ WaitForGoSignal (UMS-2 bekle)
├─ DriveWithRecovery
│   └─ DriveAllSegments
│       └─ Repeat(-1) → SegmentLoop
│           ├── HasMoreSegments
│           ├── GetCurrentSegment
│           ├── ClearHandledFlags
│           ├── ExecuteSegment (tip bazlı yönlendirme)
│           │   ├── LANE_FOLLOW → SafeDrive
│           │   ├── INTERSECTION → yavaşla + SafeDrive
│           │   ├── ROUNDABOUT → boşluk bekle + SafeDrive
│           │   ├── TUNNEL → far aç + SafeDrive + far kapat
│           │   ├── PASSENGER_STOP → dur + 15sn bekle + sinyal
│           │   └─ PARKING → slot bul + park et
│           └─ AdvanceSegment (seg_index++)
│       └─ Recovery (geri git + dön)
└─ FinishMission (dur)

SafeDrive (her segment tipinde ortak)
├─ SafetyReflexes (güvenlik katmanları)
│   ├── K1: Acil durdurma (UMS-1)
│   ├── K2: Yaya kontrolü
│   ├── K3: Dinamik engel
│   ├── K4: Statik engel
│   └─ [Tur≥2 kapısı]
│       ├── K5: Trafik ışığı
│       ├── K6: DUR tabelası
│       └─ K7: Yol levhaları
└─ FollowLaneSegment (hedef git)
```

## 9. TICK MEKANİZMASI — ADIM ADIM

`tree.tickRoot()` çağrıldığında ne olur?

Tick #1:

```
MainTree → Sequence başlar
SetBlackboard: cruise_speed=1.50 → SUCCESS
LoadMission: rota yükle → SUCCESS
WaitForGoSignal: onStart() → RUNNING ← BURADA DURUYOR
(RUNNING dönünce ağaç bekler)
```

Tick #2:

```
MainTree → Sequence devam eder
WaitForGoSignal: onRunning() → SUCCESS (sinyal geldi)
DriveWithRecovery → Fallback başlar
DriveAllSegments → Repeat başlar
HasMoreSegments: 0 < 12 → SUCCESS
GetCurrentSegment: index=0, tip=LANE_FOLLOW → SUCCESS
ClearHandledFlags → SUCCESS
ExecuteSegment → Fallback başlar
IsSegmentType: LANE_FOLLOW == LANE_FOLLOW → SUCCESS
Seg_LaneFollow → Sequence başlar
SetMaxSpeed: 1.50 → SUCCESS
SafeDrive → ReactiveSequence başlar
SafetyReflexes: tüm katmanlar kontrol → SUCCESS (tehlike yok)
FollowLaneSegment: onStart() → RUNNING ← BURADA DURUYOR
```

Tick #3:

```
... (her tick'te SafetyReflexes TEKRAR kontrol edilir – güvenlik!)
FollowLaneSegment: onRunning() → RUNNING (hedefe gidiliyor)
```

Tick #N:

```
FollowLaneSegment: onRunning() → SUCCESS (hedefe ulaşıldı!)
AdvanceSegment: seg_index = 1 → SUCCESS
(Repeat yeni iterasyona geçer: HasMoreSegments kontrol...)
```

## 10. REGISTER SİSTEMİ VE CMakeLists

### Node Kayıt (register\_all\_nodes.hpp)

```
inline void registerAllNodes(BT::BehaviorTreeFactory& factory) {
    // Her C++ sınıfını bir XML ismiyle eşle:
    factory.registerNodeType<HasMoreSegments>("HasMoreSegments");
    //           ↑ C++ sınıf adı           ↑ XML'deki ID
    // XML'de <HasMoreSegments .../> yazınca bu sınıf çalışır
    // ... toplam 43 node kaydedilir
}
```

### CMakeLists.txt — Derleme

```
# Kütüphane oluştur (5 cpp dosyası birleşir)
add_library(segment_bt_nodes SHARED
    src/segment_loop_nodes.cpp
```

```
src/vehicle_control_nodes.cpp
src/traffic_logic_nodes.cpp
src/mission_utility_nodes.cpp
src/stub_nodes.cpp
)
# ROS2 bağımlılıkları
ament_target_dependencies(segment_bt_nodes
  behaviortree_cpp_v3 rclcpp geometry_msgs std_msgs
)
# Çalıştırılabilir dosya
add_executable(bt_test src/main.cpp)
target_link_libraries(bt_test segment_bt_nodes)
```

Derleme: `colcon build --packages-select robotaxi_bt` Çalıştırma: `ros2 run robotaxi_bt bt_test segment_bt.xml`

## 11. PROJE DURUM ÖZETİ

| Metrik                  | Değer              |
|-------------------------|--------------------|
| Toplam node sayısı      | 43                 |
| Tamamlanan (gerçek kod) | 16 (%37)           |
| Stub (iskelet)          | 27 (%63)           |
| XML ağacı satır sayısı  | 494                |
| HPP dosyası sayısı      | 7                  |
| CPP dosyası sayısı      | 6                  |
| Build durumu            | ✅ Başarılı, 0 hata |

### Bizim yaptığımız:

- ✅ BT XML ağacı tasarımı (tüm yarışma senaryoları)
- ✅ 16 node'un gerçek C++ kodu (hpp + cpp)
- ✅ 27 stub node iskeleti
- ✅ ROS2 publisher altyapısı
- ✅ Modüler dosya yapısı
- ✅ Build sistemi
- ✅ Dokümanlar

### Entegrasyon için beklenen:

- Algılama ekibi: YOLO/Lidar topic'leri
- Harita ekibi: segment\_map.yaml
- Nav2: NavigateToPose action server